



CIRCUITOS DIGITALES Y MICROCONTROLADORES 2025

Facultad de Ingeniería
UNLP

Programación de puertos de
Entrada/Salida

TP1

Registros de los Puertos I/O

En clase vimos como se especifican los registros de I/O

14.4.2 PORTB – The Port B Data Register

Bit	7	6	5	4	3	2	1	0
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

14.4.3 DDRB – The Port B Data Direction Register

Bit	7	6	5	4	3	2	1	0
0x04 (0x24)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

14.4.4 PINB – The Port B Input Pins Address⁽¹⁾

[illegible]

Registros de los Puertos I/O

- También vimos como se configura la dirección

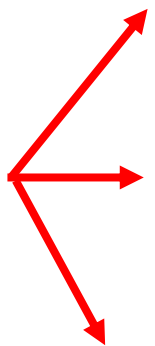
DDxn	PORTxn	I/O	Pull-up	Comment
0	0	Input	No	Tri-state (Hi-Z)
0	1	Input	Yes	Pxn will source current if ext. pulled low.
1	0	Output	No	Output Low (Sink)
1	1	Output	No	Output High (Source)

Veremos en otra clase la configuración de entrada con pull up y la estructura interna de los puertos I/O

Programando PORTS en C

¿Cómo se utilizan los registros en C?

variables uint8 que llevan
los mismos nombres que
los registros



```
DDRB= 0b01010110;
```

```
PORTB= 0xC3;
```

```
mi_var = PINB; //leer el registro de entrada y  
copiarlo a una variable
```

La asociación entre las variables y los registros se realiza en el
archivo de cabecera `io.h`

Programando PORTS en C

Uso de registros I/O en AVR - GCC

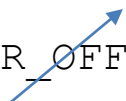
- Por ejemplo: Declaración del DDRB ([io328p.h](#))

```
#define DDRB __SFR_IO8(0x04)
```

- Macros para los SFR -Special Function Register ([sfr_defs.h](#)):

Vale 0 para
Registros I/O

```
■ #define __SFR_IO8(io_addr) __MMIO_uint8_t(io_addr + __SFR_OFFSET)
```



- Luego :

```
■ #define __MMIO_uint8_t(mem_addr) (*(volatile uint8_t *) (mem_addr))
```

- Por lo tanto en el espacio I/O:

```
DDRB es *( (volatile uint8_t *) 0x04 )
```

Programando PORTS en C

Ejemplo: escribir un valor 0xAA al PORTD

```
#include <avr/io.h>
```

← Incluye cabecera

```
int main ()  
{
```

← Función principal

```
    DDRD = 0xFF;  DDRD=11111111
```

```
    PORTD = 0xAA; PORTD=10101010
```

← Inicialización

```
    while (1);
```

← Bucle infinito

```
    return 0;
```

← Fin función principal
(nunca se alcanza)

```
}
```

Programando PORTS en C

Ejemplo: Leer, modificar y escribir un PORT

```
#include <avr/io.h>
```

← Incluye cabecera

```
int main ()
```

← Función principal

```
{
```

```
    DDRB = 0xFF;
```

← Inicialización

```
    while (1){
```

← Bucle infinito

```
        PORTD = PORTD+1;
```

← Operaciones: Leer-sumar-escribir

```
    }
```

```
    return 0;
```

← Fin función principal
(nunca se alcanza)

```
}
```

Programando PORTS en C

Ejemplo: transferir entradas a salidas

```
#include <avr/io.h>

int main ()
{
    DDRB = 0x00;
    DDRC = 0xFF;

    while (1){
        PORTC = PINB;
    }
    return 0;
}
```

Programando PORTS en C

Ejemplo: obtener PINB + PINC y escribir el resultado en PORTD.

```
#include <avr/io.h>
```

```
int main ()
```

```
{
```

```
    DDRB = 0x00;
```

Todos los bits del PORTB como entradas

```
    DDRC = 0x00;
```

Todos los bits del PORTC como entradas

```
    DDRD = 0xFF;
```

Todos los bits del PORTD como salidas

```
    while (1)
```

```
        PORTD = PINB + PINC;
```

```
        return 0;
```

```
}
```

Programando PORTS en C

Ejemplo: visualizar un contador de 8 bits en un PORT

```
#include <avr/io.h>

int main ()
{
    unsigned char z;
    DDRB = 0xFF;
    for(z=0; z<256; z++){
        PORTB=z;
    }
    while (1);
    return 0;
}
```

Resultado:

PORTB = 0x00, 0x01, 0x02, ... , 0xFF

Programando PORTS en C

Ejemplo: escribir caracteres ASCII en el PORTB

```
#include <avr/io.h>
int main ()
{
    unsigned char myList[]="012345ABCD";
    unsigned char z;
    DDRB = 0xFF;
    for(z=0;z<10;z++){
        PORTB=myList[z];
    }
    while (1);
    return 0;
}
```

¿qué representan los bits en el PORTB?

Recordemos que '0'=48=0b00110000

Programando PORTS en C

Ejemplo: Trabajando con números con signo (ca2)

```
#include <avr/io.h>

int main ()
{
    char num[]={-4,-3,-2,-1,0,1,2,3,4};
    unsigned char z;
    DDRB = 0xFF;
    for(z=0;z<9;z++){
        PORTB=num[z];
    }
    while (1);
    return 0;
}
```

Resultado:

PORTB = 0xFC, 0xFD, 0xFE, 0xFF, 0x00, 0x01, 0x02, 0x03, 0x04

Programando PORTS en C

Ejemplo: Conversión Binario-Decimal

```
#include <avr/io.h>
int main ()
{
    unsigned char x, binbyte, d1, d2, d3;
    DDRB = DDRC = DDRD = 0xFF;

    binbyte=0xFD;
    x=binbyte/10;
    d1=binbyte%10;
    d2=x%10;
    d3=x/10;
    PORTB=d1;
    PORTC=d2;
    PORTD=d3;
    while (1);
    return 0;
}
```

← dato inicial

0xFD = 253

← Unidad

Unidad = 3 = 0b00000011

← decena

Decena = 5 = 0b00000101

← centena

Centena = 2 = 0b00000010

Programando PORTS en C

¿Cómo accedemos a un solo terminal I/O del puerto B ?

¿Cómo modificamos 1bit sin alterar el resto del contenido?

14.4.2 PORTB – The Port B Data Register

Bit	7	6	5	4	3	2	1	0
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

14.4.3 DDRB – The Port B Data Direction Register

Bit	7	6	5	4	3	2	1	0
0x04 (0x24)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

DDRB

Terminal
PB0

14.4.4 PINB – The Port B Input Pins Address⁽¹⁾

[illegible]

Operadores lógicos bit a bit

Table 7-3: Bit-wise Logic Operators for C

		AND	OR	EX-OR	Inverter
A	B	A&B	A B	A^B	Y=~B
0	0	0	0	0	1
0	1	0	1	1	0
1	0	0	1	1	
1	1	1	1	0	

AND

```
1110 1111
& 0000 0001
-----
0000 0001
```

OR

```
1110 1111
| 0000 0001
-----
1110 1111
```

NOT

```
~ 1110 1011
-----
0001 0100
```

Operadores de desplazamiento

data >> number of bits to be shifted right

data << number of bits to be shifted left

1110 0000 >> 3

0001 1100

0000 0001 << 2

0000 0100

Operaciones Lógicas con bits

Ejemplo: Trabajando con operadores lógicos a nivel de bits

```
#include <avr/io.h>

int main ()
{
    DDRB = 0xFF;

    while (1){

        PORTB = PORTB | 0b00010000;
        PORTB = PORTB & 0b11101111;
    }
    return 0;
}
```

Operaciones Lógicas con bits

Ejemplo: Trabajando con operadores lógicos a nivel de bits

```
#include <avr/io.h>
```

```
int main ()
```

```
{
```

```
    DDRB =0xFF;
```

```
    DDRC = 0x00;
```

```
    while (1){
```

```
        if(PINC & 0b00100000){
```

```
            PORTB=0x55;
```

```
        }
```

```
        else{
```

```
            PORTB=0xAA;
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

```
PINC  XXXX XXXX  
& 0010 0000
```

```
00x0 0000
```

¿resultados
posibles?

Simplificaciones Lógicas con I/O

Establecer un 1 lógico en el bit 4 del PORTB

```
XXXX XXXX
| 0001 0000
-----
xxx1 xxxx
```

Otra forma de escribir la mascara:

```
XXXX XXXX
| 1 << 4
-----
xxx1 xxxx
```

```
PORTB |= (1<<4); //set bit 4 (5th bit) of PORTB
```

Notar que de esta manera no se modifican los otros bits del mismo registro

Simplificaciones Lógicas con I/O

Establecer un 0 lógico en el bit 4 del PORTB

```
      XXXX  XXXX
&     1110 1111
-----
      xxx0  xxxx
```

Otra forma de escribir la mascara:

```
      XXXX  XXXX
&     ~(1 << 4)
-----
      xxx0  xxxx
```

```
PORTB &= ~(1<<4); //clear bit 4 (5th bit) of PORTB
```

Simplificaciones Lógicas con I/O

¿está el bit 5 de PINC en 1?

PINC XXXX XXXX
& 0010 0000 == (1 << 5)

 00x0 0000
 x puede ser 0 ó 1

```
if( ((PINC & (1<<5)) != 0)    //check bit 5 (6th bit)
```

o directamente:

```
if(((PINC & (1<<5)))    //check bit 5 (6th bit)
```

Operaciones sobre registros I/O

- Resumiendo: operaciones con “<<”

```
X |= (1 << Bitnumber);    // setear un bit de la variable x, SIN AFECTAR EL RESTO
X &= ~(1 << Bitnumber);   // borrar un bit de la variable x, SIN AFECTAR EL RESTO
```

- Ejemplo:

```
#include <avr/io.h>
...
PORTD |= (1 << 2);        //setear bit 2 del port D
PORTD &= ~(1 << 2);       //borrar bit 2 del port D
```

- Utilizando los macros **Bitnumber** definidos en <avr/io.h>

```
#include <avr/io.h>
...
DDRD &= ~( (1<<PORTD0) | (1<<PORTD3) );    /* Setear PD0 y PD3 como inputs */
PORTD |= (1<<PORTD0) | (1<<PORTD3);        /* Habilitar sus Pull UP s internos */
```